23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <sys/time.h>
#include <string.h>

#define PIPELINE_COUNT 3

/* ------------------------------------------------------
   Calculate elapsed time
   ------------------------------------------------------ */
double getTimeDifference(struct timeval start,
                         struct timeval end) {

    double seconds =
        (double)(end.tv_sec - start.tv_sec);

    double microseconds =
        (double)(end.tv_usec - start.tv_usec) / 1000000.0;

    return seconds + microseconds;
}

/* ------------------------------------------------------
   Run one 3-stage pipeline
   ------------------------------------------------------ */
int runPipeline(int jobNumber) {

    int pipe1[2];
    int pipe2[2];

    pid_t p1, p2, p3;

    printf("\n[Job %d] Starting pipeline...\n",
           jobNumber);

    /*
     * Create first pipe.
     */
    if (pipe(pipe1) == -1) {

        perror("pipe1");
        return 0;
    }

    /*
     * Create second pipe.
     */
    if (pipe(pipe2) == -1) {

        perror("pipe2");

        close(pipe1[0]);
        close(pipe1[1]);

        return 0;
    }
```

```
    /* ------------------------------------------------------
       Process 1
       seq 1 1000
       ------------------------------------------------------ */

    p1 = fork();

    if (p1 == -1) {

        perror("fork process 1");
        return 0;
    }

    if (p1 == 0) {

        /*
         * stdout -> pipe1
         */
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);

        close(pipe2[0]);
        close(pipe2[1]);

        execlp("seq", "seq", "1", "1000", NULL);

        perror("seq failed");
        exit(1);
    }


    /* ------------------------------------------------------
       Process 2
       sort
       ------------------------------------------------------ */

    p2 = fork();

    if (p2 == -1) {

        perror("fork process 2");
        return 0;
    }

    if (p2 == 0) {

        /*
         * stdin <- pipe1
         */
        dup2(pipe1[0], STDIN_FILENO);

        /*
         * stdout -> pipe2
         */
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
```

```
         * stdin <- pipe1
         */
        dup2(pipe1[0], STDIN_FILENO);

        /*
         * stdout -> pipe2
         */
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);

        close(pipe2[0]);
        close(pipe2[1]);

        execlp("sort", "sort", NULL);

        perror("sort failed");
        exit(1);
    }


    /* ------------------------------------------------------
       Process 3
       wc -l
       ------------------------------------------------------ */

    p3 = fork();

    if (p3 == -1) {

        perror("fork process 3");
        return 0;
    }

    if (p3 == 0) {

        /*
         * stdin <- pipe2
         */
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);

        close(pipe2[0]);
        close(pipe2[1]);

        /*
         * Final output goes to terminal.
         */
        execlp("wc", "wc", "-l", NULL);

        perror("wc failed");
        exit(1);
    }


    /* ------------------------------------------------------
       Parent closes all pipe descriptors
```

```
    /* ------------------------------------------------------
       Parent closes all pipe descriptors
       ------------------------------------------------------ */

    close(pipe1[0]);
    close(pipe1[1]);

    close(pipe2[0]);
    close(pipe2[1]);


    /* ------------------------------------------------------
       Wait for all pipeline processes
       ------------------------------------------------------ */

    int status1;
    int status2;
    int status3;

    waitpid(p1, &status1, 0);
    waitpid(p2, &status2, 0);
    waitpid(p3, &status3, 0);


    /*
     * Detect failures.
     */
    if (!WIFEXITED(status1) ||
        WEXITSTATUS(status1) != 0) {

        printf("[Job %d] Process 1 failed.\n",
               jobNumber);

        return 0;
    }

    if (!WIFEXITED(status2) ||
        WEXITSTATUS(status2) != 0) {

        printf("[Job %d] Process 2 failed.\n",
               jobNumber);

        return 0;
    }

    if (!WIFEXITED(status3) ||
        WEXITSTATUS(status3) != 0) {

        printf("[Job %d] Process 3 failed.\n",
               jobNumber);

        return 0;
    }

    printf("[Job %d] Pipeline completed successfully.\n",
           jobNumber);

    return 1;
}
```

```c
/* ------------------------------------------------------
   Main
   ------------------------------------------------------ */

int main() {

    struct timeval start;
    struct timeval end;

    struct rusage usage;

    int successfulJobs = 0;
    int failedJobs = 0;

    printf("Large Pipeline Performance Monitor\n");
    printf("----------------------------------\n");

    printf("Pipeline:\n");
    printf("seq 1 1000 | sort | wc -l\n");

    printf("\nLaunching %d pipeline jobs...\n",
           PIPELINE_COUNT);


    /*
     * Start performance timer.
     */
    gettimeofday(&start, NULL);

    /* ------------------------------------------------------
       Launch multiple pipeline jobs
       ------------------------------------------------------ */

    for (int i = 1; i <= PIPELINE_COUNT; i++) {

        if (runPipeline(i)) {

            successfulJobs++;

        } else {

            failedJobs++;
        }
    }


    /*
     * Stop performance timer.
     */
    gettimeofday(&end, NULL);

    /*
     * Get resource usage of child processes.
     */
    if (getrusage(RUSAGE_CHILDREN, &usage) == -1) {
```

```c
    /*
     * Get resource usage of child processes.
     */
    if (getrusage(RUSAGE_CHILDREN, &usage) == -1) {

        perror("getrusage");

        return 1;
    }


    double elapsed =
        getTimeDifference(start, end);


    /* ------------------------------------------------------
       Performance Report
       ------------------------------------------------------ */

    printf("\n====================================\n");
    printf("         PERFORMANCE REPORT\n");
    printf("====================================\n");

    printf("Total Jobs        : %d\n",
           PIPELINE_COUNT);

    printf("Successful Jobs   : %d\n",
           successfulJobs);

    printf("Failed Jobs       : %d\n",
           failedJobs);

    printf("Execution Time    : %.6f seconds\n",
           elapsed);

    printf("User CPU Time     : %ld.%06ld seconds\n",
           usage.ru_utime.tv_sec,
           usage.ru_utime.tv_usec);

    printf("System CPU Time   : %ld.%06ld seconds\n",
           usage.ru_stime.tv_sec,
           usage.ru_stime.tv_usec);

    printf("Maximum Memory    : %ld KB\n",
           usage.ru_maxrss);

    printf("====================================\n");


    /*
     * Stability check.
     */
    if (failedJobs == 0) {

        printf("Stability Test    : PASSED\n");
        printf("All pipelines completed successfully.\n");

    } else {

        printf("Stability Test    : FAILED\n");
```

```
            usage.ru_stime.tv_sec,
            usage.ru_stime.tv_usec);

    printf("Maximum Memory    : %ld KB\n",
            usage.ru_maxrss);

    printf("====================================\n");


    /*
     * Stability check.
     */
    if (failedJobs == 0) {

        printf("Stability Test    : PASSED\n");
        printf("All pipelines completed successfully.\n")

    } else {

        printf("Stability Test    : FAILED\n");
        printf("Some pipeline jobs failed.\n");
    }


    /*
     * Performance analysis.
     */
    if (elapsed < 1.0) {

        printf("Performance       : GOOD\n");

    } else {

        printf("Performance       : REVIEW REQUIRED\n");
    }


    printf("====================================\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:        nano Skill23a.c
sanjana3027@DESKTOP-QJF5AGU:        nano Skill23a.c
sanjana3027@DESKTOP-QJF5AGU:        gcc Skill23a.c -o s23a
sanjana3027@DESKTOP-QJF5AGU:        ./s23a
Large Pipeline Performance Monitor
-----------------------------------
Pipeline:
seq 1 1000 | sort | wc -l

Launching 3 pipeline jobs...

[Job 1] Starting pipeline...
1000
[Job 1] Pipeline completed successfully.

[Job 2] Starting pipeline...
1000
[Job 2] Pipeline completed successfully.

[Job 3] Starting pipeline...
1000
[Job 3] Pipeline completed successfully.

====================================
        PERFORMANCE REPORT
====================================
Total Jobs       : 3
Successful Jobs  : 3
Failed Jobs      : 0
Execution Time   : 0.021224 seconds
User CPU Time    : 0.015458 seconds
System CPU Time  : 0.016926 seconds
Maximum Memory   : 8540 KB
====================================
Stability Test   : PASSED
All pipelines completed successfully.
Performance      : GOOD
====================================
sanjana3027@DESKTOP-QJF5AGU:
```

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/*
 * SKILL 24:
 * Architecture Documentation and System Call Demonstration
 *
 * Architecture:
 *
 *         MAIN PROCESS
 *              |
 *              | fork()
 *              v
 *         CHILD PROCESS
 *              |
 *              | execvp()
 *              v
 *         COMMAND EXECUTION
 *              |
 *              v
 *         COMMAND OUTPUT
 *
 * Parent waits using waitpid()
 */

void documentArchitecture() {

    printf("\n=========================================\n");
    printf("          SYSTEM ARCHITECTURE\n");
    printf("=========================================\n");

    printf("1. Main Process\n");
    printf("         |\n");
    printf("         | fork()\n");
    printf("         v\n");
    printf("2. Child Process\n");
    printf("         |\n");
    printf("         | execvp()\n");
    printf("         v\n");
    printf("3. Command Execution\n");
    printf("         |\n");
    printf("         v\n");
    printf("4. Command Output\n");
    printf("         |\n");
    printf("         v\n");
    printf("5. Parent waits using waitpid()\n");
}

void executeCommand() {

    pid_t pid;
    int status;

    printf("\n=======================================\n");
    printf("            EXECUTION PATH\n");
    printf("=======================================\n");

    printf("Parent: Creating child process...\n");

    /*
     * System Call 1: fork()
     */
```

```
    /*
     * System Call 1: fork()
     */
    pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    /*
     * CHILD PROCESS
     */
    if (pid == 0) {

        printf("Child : Process created successfully.\n");
        printf("Child : PID = %d\n", getpid());

        printf("Child : Executing 'ls' command...\n");

        /*
         * System Call 2:
         * execvp()
         */
        char *args[] = {"ls", "-l", NULL};

        execvp(args[0], args);

        /*
         * execvp() only returns
         * when execution fails.
         */
        perror("execvp");

        exit(1);
    }

    /*
     * PARENT PROCESS
     */
    else {

        printf("Parent: Child PID = %d\n", pid);

        printf("Parent: Waiting for child...\n");

        /*
         * System Call 3:
         * waitpid()
         */
        if (waitpid(pid, &status, 0) == -1) {

            perror("waitpid");
            return;
        }

        if (WIFEXITED(status)) {

            printf("Parent: Child exited normally.\n");
            printf("Parent: Exit status = %d\n",
                   WEXITSTATUS(status));
```

```c
        if (WIFEXITED(status)) {

            printf("Parent: Child exited normally.\n");
            printf("Parent: Exit status = %d\n",
                   WEXITSTATUS(status));

        } else {

            printf("Parent: Child did not exit normally.\n");
        }
    }
}

void documentSystemCalls() {

    printf("\n=========================================\n");
    printf("          SYSTEM CALLS USED\n");
    printf("=========================================\n");

    printf("fork()    -> Creates a child process\n");
    printf("execvp()  -> Executes a new program\n");
    printf("waitpid()-> Parent waits for child\n");
    printf("getpid()  -> Retrieves process ID\n");
}

void documentDesignDecisions() {

    printf("\n=========================================\n");
    printf("          DESIGN DECISIONS\n");
    printf("=========================================\n");

    printf("1. Parent creates a child using fork().\n");
    printf("2. Child executes the command using execvp().\n");
    printf("3. Parent waits using waitpid().\n");
    printf("4. Exit status is checked for reliability.\n");
    printf("5. Functions are separated for clarity.\n");
}

int main() {

    printf("SYSTEM ARCHITECTURE DOCUMENTATION DEMO\n");
    printf("=======================================\n");

    documentArchitecture();

    documentSystemCalls();

    documentDesignDecisions();

    executeCommand();

    printf("\n=========================================\n");
    printf("        DOCUMENTATION REVIEW\n");
    printf("=========================================\n");

    printf("Architecture  : Documented\n");
    printf("Flow          : Explained\n");
    printf("System Calls  : Identified\n");
    printf("Execution     : Verified\n");
    printf("Design        : Recorded\n");

    printf("\nDocumentation review completed successfully.\n");
```

```c
    printf("Design        : Recorded\n");

    printf("\nDocumentation review completed successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:        [LL$ nano Skill23b.c
sanjana3027@DESKTOP-QJF5AGU:        [LL$ gcc Skill23b.c -o s23b
sanjana3027@DESKTOP-QJF5AGU:        [LL$ ./s23b
SYSTEM ARCHITECTURE DOCUMENTATION DEMO
---------------------------------------

=========================================
        SYSTEM ARCHITECTURE
=========================================
1. Main Process
        |
        | fork()
        v
2. Child Process
        |
        | execvp()
        v
3. Command Execution
        |
        v
4. Command Output
        |
        v
5. Parent waits using waitpid()

=========================================
          SYSTEM CALLS USED
=========================================
fork()    -> Creates a child process
execvp()  -> Executes a new program
waitpid()-> Parent waits for child
getpid()  -> Retrieves process ID

=========================================
          DESIGN DECISIONS
=========================================
1. Parent creates a child using fork().
2. Child executes the command using execvp().
3. Parent waits using waitpid().
4. Exit status is checked for reliability.
5. Functions are separated for clarity.

=========================================
          EXECUTION PATH
=========================================
Parent: Creating child process...
Parent: Child PID = 4295
Parent: Waiting for child...
Child : Process created successfully.
Child : PID = 4295
Child : Executing 'ls' command...
total 968
-rwxr-xr-x 1 sanjana3027@DESKTOP sanjana3027@DESKTOP-QJF5AGU:      16:19 S2a
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16776 Oct  3 16:24 S2b
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16776 Oct  3 16:35 S3a
-rwxr-xr-x 1 ketana_sindhu sanjana3027@DESKTOP-QJF5AGU:      16:41 S3b
-rw-r--r-- 1 sanjana3027@DESKTOP-QJF5AGU:                    17:18 Skill6
-rw-r--r-- 1 ketana_sindhu ketana_sindhu   316 Aug 24 06:01 Skill1
-rw-r--r-- 1 ketana_sindhu ketana_sindhu  2695 Oct  4 03:04 Skill1
```

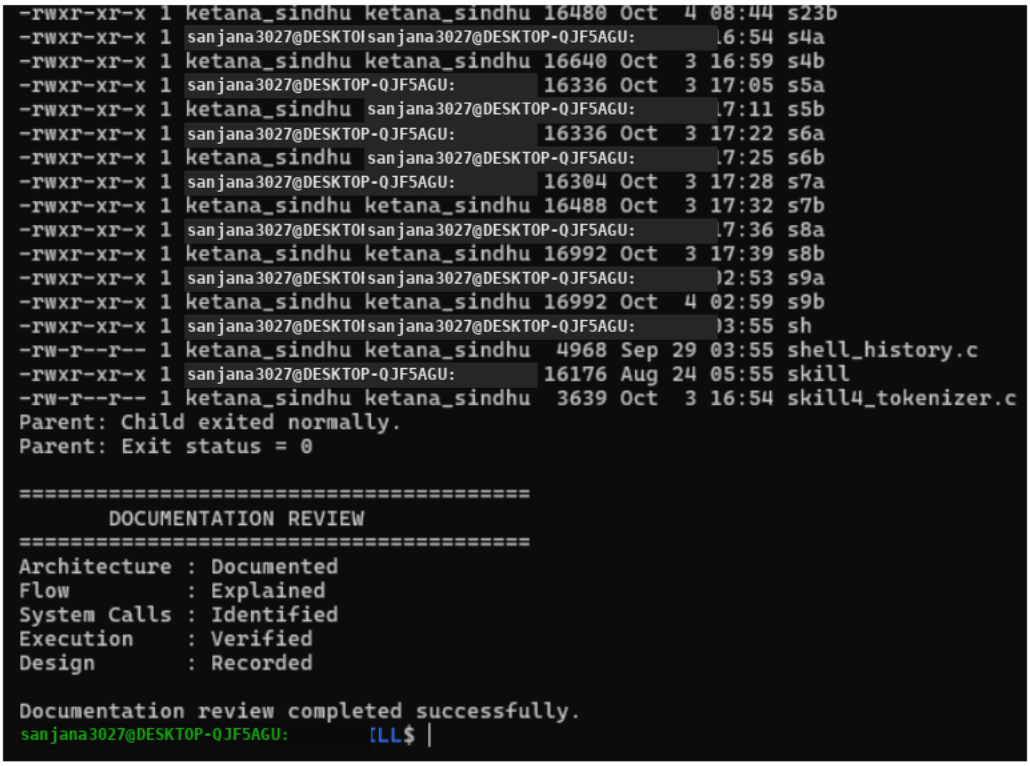
.

.

.

.

.


-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16480 Oct 4 08:44 s23b
-rwxr-xr-x 1 sanjana3027@DESKTOP sanjana3027@DESKTOP-QJF5AGU: 16:54 s4a
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16640 Oct 3 16:59 s4b
-rwxr-xr-x 1 sanjana3027@DESKTOP-QJF5AGU: 16336 Oct 3 17:05 s5a
-rwxr-xr-x 1 ketana_sindhu sanjana3027@DESKTOP-QJF5AGU: 17:11 s5b
-rwxr-xr-x 1 sanjana3027@DESKTOP-QJF5AGU: 16336 Oct 3 17:22 s6a
-rwxr-xr-x 1 ketana_sindhu sanjana3027@DESKTOP-QJF5AGU: 17:25 s6b
-rwxr-xr-x 1 sanjana3027@DESKTOP-QJF5AGU: 16304 Oct 3 17:28 s7a
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16488 Oct 3 17:32 s7b
-rwxr-xr-x 1 sanjana3027@DESKTOP sanjana3027@DESKTOP-QJF5AGU: 17:36 s8a
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16992 Oct 3 17:39 s8b
-rwxr-xr-x 1 sanjana3027@DESKTOP sanjana3027@DESKTOP-QJF5AGU: 02:53 s9a
-rwxr-xr-x 1 ketana_sindhu ketana_sindhu 16992 Oct 4 02:59 s9b
-rwxr-xr-x 1 sanjana3027@DESKTOP sanjana3027@DESKTOP-QJF5AGU: 03:55 sh
-rw-r--r-- 1 ketana_sindhu ketana_sindhu 4968 Sep 29 03:55 shell_history.c
-rwxr-xr-x 1 sanjana3027@DESKTOP-QJF5AGU: 16176 Aug 24 05:55 skill
-rw-r--r-- 1 ketana_sindhu ketana_sindhu 3639 Oct 3 16:54 skill4_tokenizer.c
Parent: Child exited normally.
Parent: Exit status = 0
=====================================
DOCUMENTATION REVIEW
=====================================
Architecture : Documented
Flow : Explained
System Calls : Identified
Execution : Verified
Design : Recorded
Documentation review completed successfully.
sanjana3027@DESKTOP-QJF5AGU: [LL$